IC PRÁCTICA 2

Daniel Heckstall-Smith Montero

Jordi Bodí Soriano

Alejandro Algarra Galdón

Índice

I. Objetivos..... 3
II. Procedencia del código y modificaciones..... 4
III. Descripción de la aplicación y del caso de prueba de referencia..... 5
IV. Grafo de dependencias y análisis de las oportunidades de paralelismo de datos y funcional..... 6
V. Estudio de datos y de los accesos a memoria..... 8
VI. Metodología de medición, tablas, gráficas y análisis del rendimiento secuencial. 9
VII. Opciones de compilación probadas y evidencias de autovectorización..... 13
VIII. Justificación de por qué la aplicación es una buena candidata para su paralelización..... 15
IX. Propuesta razonada de una o varias arquitecturas paralelas adecuadas..... 16
X. Resolución de problemas..... 18
XI. Conclusiones y Bibliografía..... 19

I. Objetivos

En esta práctica vamos a coger una aplicación que está programada de forma totalmente secuencial, es decir, sin ningún tipo de hilo ni estructura pensada para ejecutarse en paralelo, y vamos a estudiarla a fondo para ver si merece la pena paralelizarla y de qué manera habría que hacerlo. Cabe aclarar que en esta práctica NO vamos a programar la versión paralela, sino que nos vamos a centrar en entender cómo funciona el programa, medir cuánto tarda y de qué depende ese tiempo, y a partir de ahí proponer una forma de paralelizarlo con la que se consiga una mejora de rendimiento que de verdad compense.

Para ello hemos elegido una versión casera del Conecta 4 programada en C por Jordi Bodí, uno de los integrantes del grupo, en la que juegas contra la máquina. Para decidir dónde poner su siguiente ficha, la máquina utiliza el algoritmo Minimax con poda alfa-beta, que consiste en simular todas las jugadas posibles tanto suyas como del jugador hasta una cierta profundidad (en el código la llamamos umbral), y puntuar los tableros a los que se llega mediante una heurística que busca patrones favorables y desfavorables (tres en raya con un hueco libre, diagonales casi completas, formas en “7”...), quedándose al final con la columna que mejor puntuación le da.

Lo que hace que este programa sea interesante para la asignatura es que su coste crece de manera exponencial con el umbral, ya que en cada nivel del árbol hay hasta 7 columnas posibles, por lo que sin la poda el número de tableros a analizar sería del orden de 7 elevado al umbral. Esto es teniendo en cuenta que el tablero es pequeño, el Conecta 4 tiene alrededor de 4,5 billones de posiciones distintas (el juego se resolvió en 1988, y con juego perfecto gana siempre el que empieza poniendo en la columna central), por lo que analizar la partida hasta el final en cada jugada es directamente imposible, y la máquina se tiene que conformar con mirar sólamente unos cuantos movimientos por delante.

Además es un problema que se presta a los dos tipos de paralelismo necesarios para la práctica.

Por un lado tiene paralelismo de datos, ya que cada una de las 7 columnas que puede elegir la máquina genera una rama del árbol que no depende de las demás, por lo que se podrían explorar al mismo tiempo. Y, por otro lado, tiene paralelismo funcional, ya que existen elementos en el sistema que detienen la ejecución del código mientras se pueden estar adelantando procesos necesarios, como las llamadas de heurísticas.

Por tanto, los objetivos de esta práctica son, por una parte, analizar la estructura del programa y las dependencias entre sus funciones para ver qué partes se pueden ejecutar a la vez y cuáles no, y por otra, medir cómo se comporta la versión secuencial al cambiar el umbral y las opciones de compilación de gcc (mirando también si el compilador llega a vectorizar alguna parte por su cuenta). Con todo eso, lo último que haremos será proponer qué tipo de arquitectura paralela nos conviene más para este problema, y estimar qué ganancia y eficiencia podríamos esperar, cosa que comprobaremos de verdad cuando tengamos hecha la versión paralela.

II. Procedencia del código y modificaciones

Para la realización del paralelismo, tanto funcional cómo de datos, hemos elegido una aplicación personal de Conecta 4, que consiste en jugar contra una IA que analiza los casos favorables y desfavorables para obtener un valor resultado de los estados finales analizados en cada posible movimiento, tanto de la IA como del jugador.

En resumen, consiste en un algoritmo Minimax con poda alpha-beta.

Esta aplicación contiene una gran cantidad de bucles for en una única función, por lo que aplicaremos un sistema de hilos que ejecute cada bucle for analizando los 4 en raya correspondientes.

Cabe destacar que hemos tenido que aplicar ciertas modificaciones al código para que las pruebas sean las deseadas, siendo las más destacables la necesidad de introducir todos los movimientos a la vez (para evitar intervención humana durante la recopilación de pruebas), así como el uso de estructura clock(); para obtener los tiempos de ejecución de la forma deseada. Todo esto será detallado más adelante.

III. Descripción de la aplicación y del caso de prueba de referencia

Descripción de la aplicación: La aplicación desarrollada en lenguaje C implementa el juego Conecta 4, apoyándose en el algoritmo de inteligencia artificial Minimax para la toma de decisiones. El programa recibe un estado del tablero y explora las posibles jugadas futuras mediante la construcción de un árbol de decisiones. Utilizando una función de evaluación heurística, el algoritmo asigna una puntuación a los distintos estados de la partida para determinar cuál es el movimiento óptimo.

Caso de prueba de referencia: Tal y como se requiere, la aplicación debe poder ejecutarse sin intervención del usuario durante la fase de medición de rendimiento. Para lograrlo, hemos definido un caso de prueba cerrado y completamente determinista basado en parámetros iniciales.

Durante la evaluación del árbol Minimax, es muy frecuente encontrar varios movimientos que resultan "igual de buenos" al obtener la misma puntuación heurística. Para decidir entre ellos, el programa está diseñado en C utilizando la función rand() para desempatar. Para asegurar que este comportamiento no afecte a la reproducibilidad de las mediciones, el caso de prueba se inicializa con una semilla predefinida con valor 999 (srand(999)).

El uso de esta semilla fija garantiza que la secuencia de empates resueltos por rand() sea idéntica en cada ejecución. Esto define una forma perfectamente reproducible de ejecutar el programa de principio a fin sin intervención manual, permitiendo medir los tiempos con precisión.

Los parámetros de configuración de nuestro caso de prueba son los siguientes:

- Semilla de aleatoriedad: 999.
- Profundidad de búsqueda: *9 inicialmente y cada 2-3 turnos aumenta en 1*
- Estado inicial del tablero: vacío

IV. Grafo de dependencias y análisis de las oportunidades de paralelismo de datos y funcional

Para resolver nuestro problema hemos tenido que plantear las dependencias que existían en el programa, es decir, que debe estar resuelto antes de pasar al siguiente paso o qué debe esperar a que el resto esté completado para proseguir su ejecución.

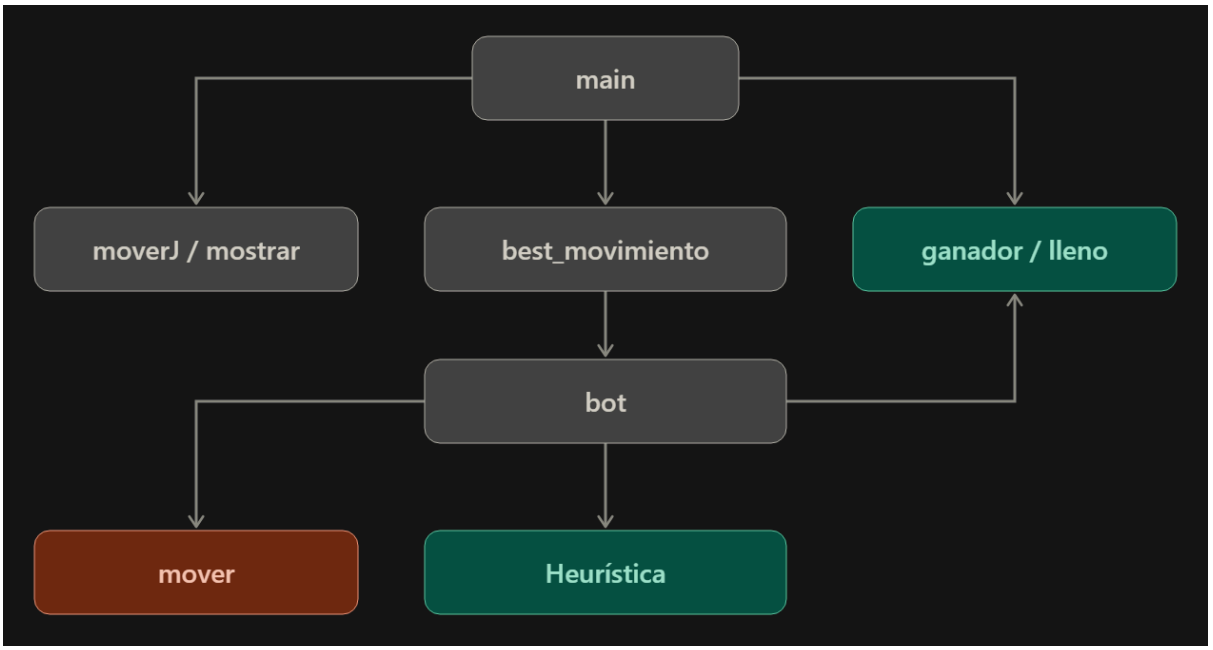
Vamos a dividir el código en dos partes principales, que son donde radicará el paralelismo.

Parte 1, Main():

En el main podemos observar las llamadas a las siguientes funciones:

Paralelizable = P, No paralelizable = NP

- Mostrar -> No modifica el mapa, sólamente lo lee, se puede realizar a la vez que otras tareas que no modifiquen el mapa. P
- Inicializar -> Crea el tablero, no realiza una función paralelizable. NP
- Mover -> Modifica una posición del tablero. NP
- MoverJ -> Modifica una posición del tablero. NP
- Best_movimiento -> Caso analizado más adelante.
- Ganador -> Comprueba sólamente las posiciones, no modifica el tablero. P
- Lleno -> Igual que ganador. P



La función main está bastante limitada con respecto al paralelismo funcional, ya que la mayoría de las funciones son independientes. Pero aquí vamos a cambiar el planteamiento del problema. Ya que estamos limitados a la no intervención del ser humano en el problema, vamos a recibir todas las entradas de golpe y dejarlas almacenadas en el buffer, lo que nos permitirá tener el problema ya planteado mientras se ejecuta el mostrar tablero, por lo que podemos empezar a calcular las posibles soluciones de la siguiente jugada MIENTRAS se está mostrando el tablero, lo cual es paralelismo funcional, ya que son dos tareas distintas (mostrar el tablero y calcular la jugada) ejecutándose a la vez. Por otro lado la heurística es un conjunto de funciones que evalúa diferentes figuras en el tablero que da una ventaja a la máquina o al jugador, todas estas funciones se podrían hacer simultáneamente aplicando paralelismo funcional y a su vez cada función de la heurística va recorriendo la matriz buscando los diferentes patrones en este caso se puede segmentar la matriz en trozos donde se aplica las mismas instrucciones para todos los datos de la matriz siendo así paralelismo de datos.

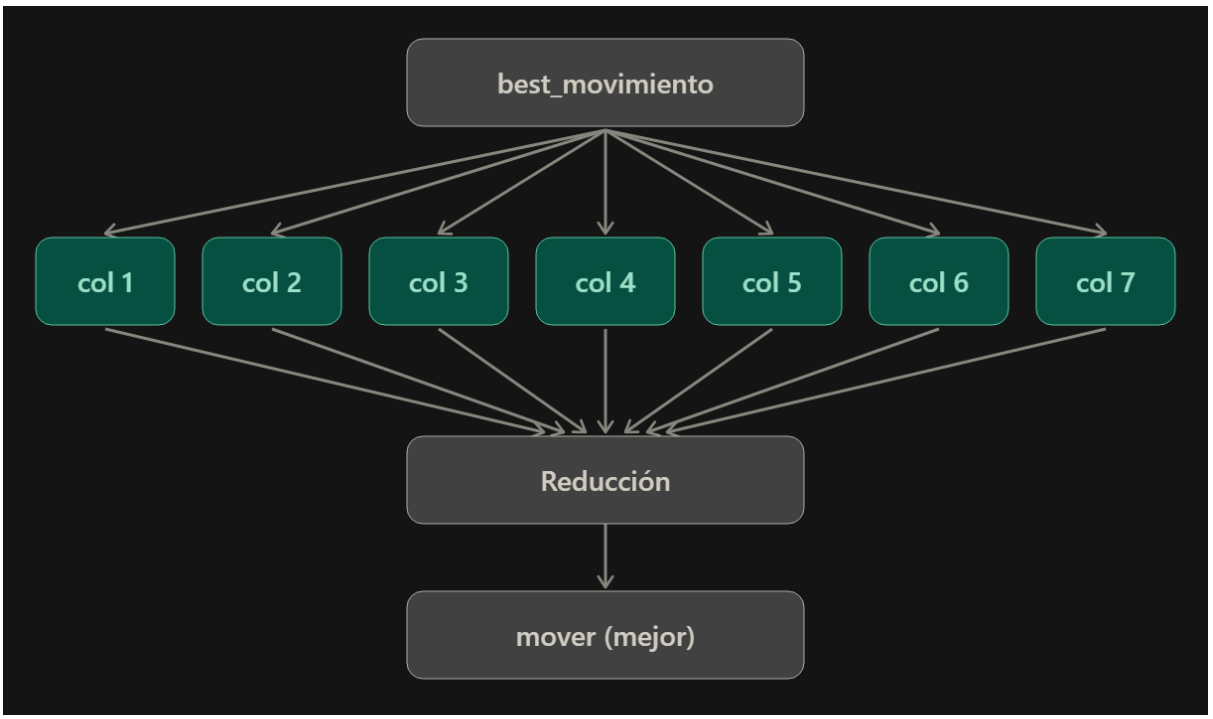
Parte 2, best_movimiento():

Best movimiento tiene una estructura por la que sus bucles for son completamente independientes los unos de los otros.

Pese a que esté atado a modificaciones actuales del código, se puede plantear una estructura independiente de tablero y realizar todas las iteraciones del bucle for por separado, dividiendo el tiempo de ejecución entre el número de columnas. Lo único que faltaría sería obtener el resultado como vuelta de todas las columnas como el mejor de ellos, llamémoslo reducción en nuestro caso. Cada columna es una rama del árbol que no depende de las demás, por lo que las 7 llamadas a bot() se pueden ejecutar a la vez, lo cual es paralelismo funcional.

En la función best movimiento podemos observar las llamadas a las siguientes funciones:

- Rebobinar (Falta ortográfica en el código, encontrada como “revovinar") -> Modifica una posición en el tablero, al igual que mover. Pero ya que sus movimientos no son definitivos para el tablero, se puede plantear hacerlos en una copia alternativa de tablero, permitiendo paralelizar todo best movimiento. P
- Mover -> Teóricamente no paralelizable, pero sufre las mismas consecuencias que rebobinar, por lo que con el supuesto planteado en rebobinar también podríamos paralelizar sin problema esta parte. P
- Bot -> Realiza todos los cálculos necesarios para aplicar el algoritmo Minimax. P



V. Estudio de datos y de los accesos a memoria

Estructuras relevantes:

- Tablero[fil][col] -> Se modifica en cada paso de ejecución y en cada decisión tomada y por tomar, por lo que es muy importante que en la paralelización cada toma de decisiones tenga la suya propia. Actualmente a ella accede el programa como una copia del original y lo modifica mediante mover.
- Alpha y beta -> Son variables que "bot" utiliza para obtener el Minimax, por lo que deberán ser atómicas al paralelizar el problema. Actualmente a ella accede mediante una copia en cada llamada de bot.
- Umbral -> Variable que utiliza el programa para darle valor a la heurística. Se deberá tratar como alpha y beta pero, ya que no se modifica hasta que no se ha tomado una decisión, no será atómica. Actualmente a ella se accede como una copia de la original y se modifica al final de ejecución.
- Res, puntuación, mejor puntuación -> Se actualizan en cada cálculo dado por "bot" en cada ejecución de best_movimiento. El resultado de la reducción deberá ser el mayor de todos los resultados devueltos, dando la opción de no volver atómico mejor puntuación. Actualmente se accede a ellos mediante la ejecución normal de best_movimiento.
- Pila de llamadas -> Ya que bot utiliza recursión, la frecuencia de llamadas a la pila es muy alta, por lo que se ha de tener en cuenta para este análisis.
- Heurística -> Son todos los datos que se utilizan para decidir cuáles son los mejores movimientos.

Como ya he comentado anteriormente, la estrategia más inteligente sería modificar tablero para que se acceda a una copia diferente en cada hilo, modificando sólamente la original una vez se haya encontrado el mejor movimiento. Además, mantener alpha y beta como variables

atómicas para evitar colisiones entre los hilos, ya que todos deberán modificarlas si el caso cumple la condición. La heurística será generada en cada hilo, por lo que no es necesario modificar su estructura actual y, con respecto al resto de variables, no será necesaria ninguna modificación para el correcto funcionamiento.

VI. Metodología de medición, tablas, gráficas y análisis del rendimiento secuencial

Antes de poder medir nada, hemos tenido que adaptar un poco el programa, ya que tal y como estaba planteado no servía para medir de forma fiable.

Por un lado el programa es interactivo (pide el modo de juego y cada jugada por teclado), y por otro el tiempo se medía con time(NULL), que sólamente tiene precisión de segundos, por lo que una jugada que tarde 0,4 segundos y otra que tarde 1,3 podían salir con el mismo tiempo o directamente con 0. Además ese tiempo incluía todo lo que pasa dentro del bucle del main, es decir, también el mostrar() del tablero, que hace un system("clear") en cada llamada (crea un proceso nuevo sólo para limpiar la pantalla), lo cual no tiene nada que ver con lo que nosotros queremos medir.

Para solucionarlo hemos hecho tres cambios intentando tocar lo mínimo el código original:

- Hemos sustituido time(NULL) por clock_gettime con CLOCK_MONOTONIC, que nos da precisión de nanosegundos
- Hemos añadido una opción para desactivar el pintado del tablero mientras se mide.
- Y por último, las jugadas del jugador las recibimos todas de golpe desde un fichero (./conecta4 < entrada.txt), tal y como explicamos en el apartado IV. Cabe aclarar que como moverJ ya descarta por sí solo las columnas que están llenas y vuelve a leer, el fichero tiene una secuencia de columnas lo suficientemente larga para que nunca se quede sin jugadas válidas, ya que si se acaba la entrada el programa se queda en un bucle infinito.

Con respecto a la medición del programa, vamos a medir el programa de tres formas distintas :

- La primera es la partida completa del caso de referencia (semilla 999, umbral inicial 9 y tablero vacío), que es lo que ejecuta make run, y de la cual guardamos tanto el tiempo total como lo que tarda cada jugada de la máquina por separado. Esta es la que usaremos para comparar las opciones de compilación en el apartado VII. Pero para estudiar cómo escala el problema no nos sirve del todo, ya que al ir subiendo el umbral durante la partida y partir cada jugada de un tablero distinto, hace que se mezclen dos efectos a la vez y no se pueda saber cuál de los dos es el que hace que tarde más.

- Por eso la segunda medición es UNA sola jugada de la máquina partiendo siempre del mismo tablero (vacío), variando el umbral desde 4 hasta [N], que es el parámetro que de verdad controla la carga de trabajo del problema. Sin la poda el árbol crecería del orden de 7^umbral, es decir, cada nivel más multiplicaría el tiempo por 7, pero gracias a la poda alfa-beta el crecimiento debería ser bastante menor. Lo representamos en una gráfica con el eje Y en escala logarítmica, ya que así un crecimiento exponencial se ve como una línea recta, y es mucho más fácil ver cuánto se multiplica el tiempo por cada nivel que añadimos.

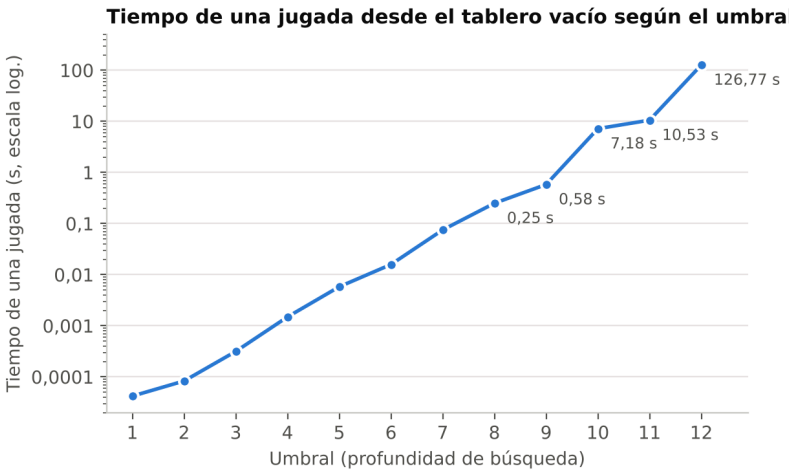
- La tercera medición, llamémosla reparto por columnas, consiste en medir por separado lo que tarda cada una de las 7 llamadas a bot() que hace best_movimiento(). Esto el enunciado no lo pide directamente, pero nos interesa mucho de cara a paralelizar, ya que como vimos en el apartado IV cada columna se puede calcular en un hilo distinto, y si unas columnas tardan bastante más que otras, el hilo más lento es el que va a marcar el tiempo total (igual que pasa con el grifo lento en los problemas de la tarea 3). Además de esto, hemos pasado el programa por gprof (compilando con -pg) para saber qué funciones son las que se llevan la mayor parte del tiempo de ejecución.

Las siguientes mediciones se han hecho en un portátil MSI GF63 thin, 2.3 GHz, gcc 13.3, -O2, con el portátil enchufado y utilizando una máquina virtual con dos CPU de procesamiento. Cada medición la hemos repetido 5 veces descartando la primera ejecución, que suele salir algo más lenta, y en las tablas ponemos la mediana junto con el mínimo y el máximo, ya que la media se ve muy afectada si alguna ejecución sale mal por cualquier proceso que se ponga a funcionar de fondo. En este apartado hemos compilado siempre con gcc -O2, y el resto de opciones de compilación las comparamos en el apartado VII con el mismo caso de prueba.

Umbral	Mediana (s)	Mín (s)	Máx (s)	Factor respecto al umbral anterior
8	0,2495	0,1947	0,2747	×3,33
9	0,5804	0,5546	0,6483	×2,33
10	7,182	6,748	7,479	×12,37
11	10,53	10,02	11,32	×1,47
12*	126,77	126,38	128,60	×12,04

Tabla 1: Una jugada desde el tablero vacío según el umbral (./conecta4 -escalado 1 12)

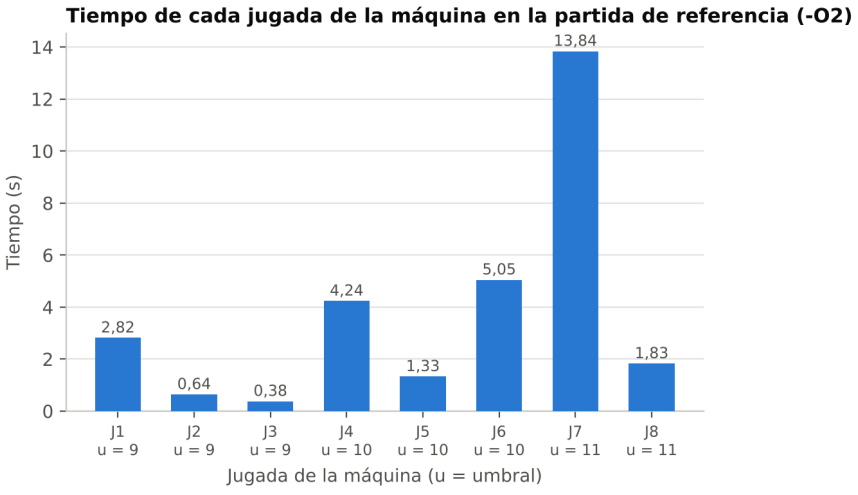
* solamente 3 ejecuciones ya que en el resto, la ejecución acababa antes



Como se puede ver en la tabla, de media cada nivel de umbral multiplica el tiempo unas 4 o 5 veces (4,1 de media entre el umbral 4 y el 12), bastante lejos del 7 que tendríamos sin poda. Pero el crecimiento no es nada regular, ya que de 9 a 10 y de 11 a 12 el tiempo se multiplica por 12, mientras que de 10 a 11 sólamente por 1,5. Contando los nodos que visita bot() (tabla 1b) se ve que con el número de nodos pasa exactamente lo mismo, y que cada nodo cuesta siempre más o menos lo mismo (entre 0,6 y 0,8 µs), por lo que estos saltos no son cosa de la máquina sino de la poda, que corta más o menos ramas según la profundidad y el orden en el que se exploran las columnas (siempre de izquierda a derecha). Aun así la poda es muy efectiva, ya que con umbral 12 bot() sólamente visita el 0,16 % de los nodos que tendría el árbol completo.

Jugada	Umbral	Mediana (s)	Mín (s)	Máx (s)	% de la partida
1	9	2,816	2,734	2,983	9,4 %
2	9	0,642	0,501	0,644	2,1 %
3	9	0,380	0,372	0,409	1,3 %
4	10	4,237	3,916	4,262	14,1 %
5	10	1,332	1,276	1,474	4,4 %
6	10	5,050	4,547	5,277	16,8 %
7	11	13,840	13,542	14,623	46,0 %
8	11	1,829	1,673	2,105	6,1 %
Total		**30,116**	29,495	30,525	100 %

Tabla 2: Tiempo de cada jugada en la partida de referencia (make run)

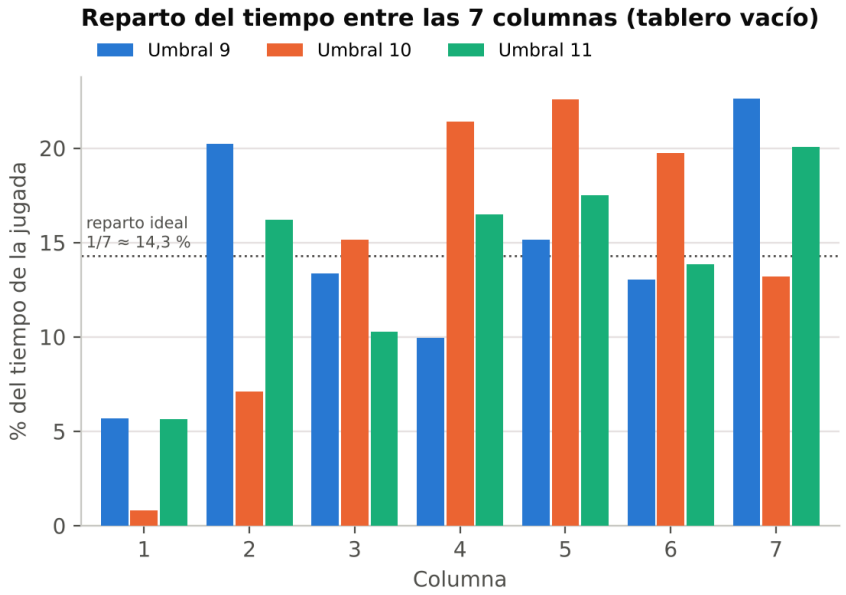


La máquina gana en su 8.ª jugada, y las 6 ejecuciones acaban con el mismo tablero. La desviación típica del total es 0,37 s, un 1,2 %.

En la partida completa se ve como el tiempo de cada jugada no depende sólamente del umbral, sino también de cómo está el tablero en ese momento, por lo que dos jugadas con el mismo umbral pueden tardar desde 1,83 hasta 13,84 segundos.

Columna	Mediana (s)	Mín (s)	Máx (s)	% del tiempo de la jugada	Nodos
1	0,0554	0,0431	0,0572	0,8 %	75.260
2	0,4958	0,4835	0,6634	7,1 %	748.906
3	1,056	0,9950	1,152	15,1 %	1.631.094
4	1,492	1,382	1,718	21,4 %	2.190.032
5	1,576	1,177	1,747	22,6 %	2.341.602
6	1,376	1,320	1,642	19,7 %	1.975.358
7	0,9203	0,8658	0,9999	13,2 %	1.284.101
Suma	**6,971**			100 %	10.246.353

Tabla 3. Reparto por columnas, umbral 10 (./conecta4 -columnas 10)



Por último, en el reparto por columnas la más rápida tarda 0,0554 s y la más lenta 1,576 s, por lo que aunque repartiéramos las 7 columnas en 7 hilos, la ganancia como mucho sería de 6,971/1,576 = 4,4, y no de 7. Con respecto a gprof (compilando con -O2 -fno-inline -pg, ya que si no el compilador mete unas funciones dentro de otras y gprof no las puede separar), la función heurística (tresEnRaya, rectángulos y compañía) se lleva alrededor del 59 % del tiempo (sólamente tresEnRaya ya el 33,5 %) y ganador() el 35 % (contando comprobacionH y comprobacionV, que son las que llama ganador), por lo que es ahí donde va a estar la mayor parte del trabajo a paralelizar. En la partida de referencia la heurística se evalúa sobre 23,5 millones de tableros..

VII. Opciones de compilación probadas y evidencias de autovectorización

Las opciones de compilación obtienen diferentes tiempos ya que las flags de optimización permiten mejorar el tiempo de ejecución del programa, teniendo en cuenta el sacrificio en compilación que este conlleva.

En nuestro caso, conocemos la facilidad de autovectorización que el programa implica, por lo que podemos deducir que el uso de una optimización en compilación podrá mejorar el rendimiento del programa, por lo que hemos realizado una prueba sobre nuestro código obteniendo los siguientes resultados (3 pruebas cada ejecución con los mismos datos):

Opciones	Mediana (s)	Mín (s)	Máx (s)	Ganancia vs -O0	Ganancia vs -O2
-O0	62,09	59,96	63,20	×1,00	×0,49
-O1	30,97	30,36	31,64	×2,01	×0,97
-O2	30,14	28,77	30,67	×2,06	×1,00
-O3	19,60	19,41	20,70	×3,17	×1,54
-Os	34,29	32,65	35,57	×1,81	×0,88
-Ofast	20,26	18,92	20,68	×3,06	×1,49

Tabla 4: Tiempos de ejecución con distintos niveles de optimización

En la tabla se puede observar que para el umbral 9 casi no se ha modificado nada, pero para el umbral 10 y 11, el tiempo de ejecución se ha visto reducido drásticamente en algunos casos.

Tipo de compilación	Tiempos de compilación
-O0	0.08 s
-O1	0.15 s
-O2	0.24 s
-O3	0.48 s
-Os	0.18 s
-Ofast	0.43 s

Tabla 5: Tiempo medio de compilación por tipo de compilación

Para demostrar la autovectorización hemos realizado una compilación -O3 con la opción foto-info-vec-optimized=vectorizados.txt, que ha almacenado los bucles que -O3 ha podido detectar vectorizables, devolviéndonos la siguiente salida:

Conecta4.c:505:19: optimized: loop vectorized using 16 byte vectors

Conecta4.c:505:19: optimized: loop vectorized using 8 byte vectors

Conecta4.c:618:18: optimized: loop vectorized using 16 byte vectors

Conecta4.c:505:19: optimized: loop vectorized using 16 byte vectors

Conecta4.c:505:19: optimized: loop vectorized using 8 byte vectors

Además, conforme hemos ido explicando a lo largo del documento, se puede observar con claridad que la responsabilidad de los bucles dentro de la función best_movimiento es fácilmente autovectorizable, permitiéndonos hacer todos los casos por separados ya que ninguno afecta a la salida del resto, pero todos los resultados se necesitan para comparar cuál obtiene un mejor valor.

VIII. Justificación de por qué la aplicación es una buena candidata para su paralelización

El algoritmo Minimax aplicado al juego Conecta 4 es un excelente candidato para su paralelización debido a que presenta una carga computacional elevada que justifica plenamente este estudio. A medida que aumenta la profundidad de búsqueda para predecir movimientos futuros, el árbol de decisiones crece de forma exponencial, convirtiéndose en un problema de coste computacional apreciable.

Esta aplicación resulta idónea porque nos permite explotar los dos tipos de paralelismo requeridos en la práctica:

- Paralelismo funcional (Búsqueda en el árbol de decisiones): Existe paralelismo funcional cuando diferentes tareas u operaciones pueden ejecutarse simultáneamente si sus dependencias lo permiten. En el algoritmo Minimax, a partir de un estado del tablero, los posibles movimientos generan ramas independientes. La exploración y evaluación de cada una de estas ramas conforma tareas diferentes que pueden solaparse y ejecutarse de manera simultánea en distintos hilos o procesos. Las dependencias sólo aparecen al final del proceso, cuando los resultados parciales de las ramas deben combinarse para propagar el valor óptimo hacia la raíz del árbol.
- Paralelismo de datos (Cálculo de la función de evaluación): Hay paralelismo de datos cuando la misma operación puede aplicarse al mismo tiempo sobre partes distintas de los datos. Para evaluar cuán buena es una posición, el programa debe analizar el estado del tablero (guardado en forma de matriz). Las comprobaciones de las líneas horizontales, verticales y diagonales para calcular la puntuación heurística pueden dividirse y procesarse simultáneamente sobre las diferentes secciones de la matriz. Además, la misma función de evaluación puede aplicarse en paralelo a múltiples matrices (los distintos estados hoja del árbol).

En conclusión, la combinación de una alta exigencia computacional que escala claramente con la profundidad del árbol, junto con la presencia evidente de paralelismo de datos en la matriz y paralelismo funcional en la exploración de las ramas, hacen que esta aplicación cumpla con todos los objetivos planteados y sea una candidata ideal para analizar su rendimiento y proponer una arquitectura paralela adecuada.

IX. Propuesta razonada de una o varias arquitecturas paralelas adecuadas

Para la paralelización del algoritmo Minimax aplicado al Conecta 4, hemos analizado varias arquitecturas paralelas, evaluando un procesador multinúcleo, una GPU y un sistema heterogéneo, junto con sus posibles ventajas y dificultades:

1. Procesador multinúcleo (CPU con memoria compartida)

Esta arquitectura es idónea para explotar el paralelismo funcional asociado a la exploración de las ramas del árbol de decisiones.

- Ventajas: Su principal ventaja es la relativa facilidad de programación (mediante hilos o directivas como OpenMP) y el uso de memoria compartida. Al compartir el mismo espacio de direcciones, la creación de los estados derivados del tablero y la propagación de los valores minimax hacia la raíz se realizan de forma muy rápida y sin latencia de red.

- Dificultades: La principal dificultad es su límite de escalabilidad. Al crecer la profundidad de búsqueda, el número de nodos aumenta exponencialmente, saturando rápidamente los hilos disponibles en un procesador de escritorio típico.

2. Unidad de Procesamiento Gráfico (GPU)

La GPU destaca por su capacidad computacional masiva, siendo ideal para el paralelismo de datos en la fase de evaluación de los tableros.

- Ventajas: Permite procesar de forma simultánea miles de matrices, aplicando la función de evaluación heurística (comprobación de 4 en raya) en paralelo sobre multitud de nodos hoja del árbol de búsqueda.

- Dificultades: El algoritmo Minimax es inherentemente recursivo y presenta bifurcaciones irregulares (divergencia de control), lo cual no se adapta bien a la arquitectura SIMT (Single Instruction, Multiple Threads) de las GPUs. Además, el coste de transferir los datos del árbol desde la memoria del host (CPU) a la memoria del dispositivo (GPU) puede llegar a anular la ganancia en velocidad.

3. Sistema heterogéneo (CPU multinúcleo + GPU)

Consideramos que esta es la opción más potente y adecuada para nuestro problema, ya que combina las fortalezas de las dos arquitecturas anteriores.

- Ventajas: Permite asignar la recursividad, el control de flujo y el paralelismo funcional de las ramas superiores a la CPU, mientras que delega la evaluación masiva de los nodos hoja a la GPU mediante paralelismo de datos. Esta división del trabajo maximiza el rendimiento global.

- Dificultades: La mayor desventaja es la alta complejidad de diseño e implementación. Requiere orquestar eficientemente el trabajo entre la CPU y la GPU, agrupar las evaluaciones en lotes (batches) para maximizar la ocupación de la tarjeta gráfica y sincronizar cuidadosamente ambos dispositivos para evitar tiempos de inactividad.

X. Resolución de problemas

Problema 1: Un grifo tarda 8 horas en llenar un depósito y otro tarda 40 horas. ¿Cuánto tardarán si abrimos los dos a la vez para llenar el depósito, inicialmente vacío?

Ya que un grifo tarda 5 veces más en llenar el depósito que el otro, siendo 6 depósitos los que estarían llenos en 40 horas:

40 horas = 6 depósitos

1 depósito = x

x = 40/6 = 6.67 horas = 6 horas y 40 minutos

Problema 2: Disponéis de dos grifos, cada uno de los cuales tarda 8 horas en llenar el depósito.

8 horas = 2 depósitos

1 depósito = x

x = 8/2 = 4 horas

Problema 3: Disponéis de dos grifos, cada uno de los cuales tarda 40 horas en llenar el depósito.

40 horas = 2 depósitos

1 depósito = x

x = 40/2 = 20 horas

Problema 4: Disponéis de tres grifos: uno que tarda 8 horas y dos que tardan 40 horas.

40 horas = 7 depósitos

1 depósito = x

x = 40/7 = 5.714 horas ≈ 5 horas y 43 minutos

Caso	Ganancia	Eficiencia
1	1,2	0,6
2	2	1
3	2	1
4	1,4	0,47

XI. Conclusiones

Como conclusión podemos obtener que paralelizar un programa es una opción que se puede plantear siempre y cuándo exista la posibilidad de realizar varias tareas de forma simultánea.

Asimismo, también existen herramientas que nos permiten conocer si nuestra aplicación va a ser compatible o no con la aplicación de paralelismo y, no menos importante, este documento recalca la importancia de hacer un análisis de datos al código para conocer si realmente esa aplicación de paralelismo marca un cambio significativo o no.

XII. Bibliografía

- **Victor Allis, *A Knowledge-based Approach of Connect-Four*, tesis de máster, Vrije Universiteit Amsterdam, 1988 (el juego resuelto)**
- **John Tromp, recuento de posiciones legales del Conecta 4 (las ~4,5 billones de posiciones)**
- **Documentación oficial de GCC, «Options That Control Optimization» (gcc.gnu.org/onlinedocs)**
- **Manuales de gprof y de Valgrind/Cachegrind**
- **Creación de grafos de dependencia y gráficas de datos asistido por Claude**
- **Repositorio origen: https://github.com/Jordi-897/Practica2_IC**